Reinforcement Learning (DSAI 402)

Lecture 5: Value Iteration and Monte Carlo Prediction

Prof. Mohamed Ghalwash
mghalwash@zewailcity.edu.eg

Zewail City University

Lecture 5, Monday 26 October 2026

Agenda

Objectives: you should be able to run value iteration on a small MDP, explain generalized policy iteration, estimate v_π by averaging the returns of played episodes and tell first-visit MC from every-visit MC.

- Finish DP: value iteration, when to stop and generalized policy iteration
- Why we need a method that works without the model
- Monte Carlo prediction: returns, first-visit and every-visit MC
- Apply it: the fourth application paper, AlphaGo
- Practice it: the bridge walk game

Part 1: Finishing Dynamic Programming

Where We Stopped: Policy Iteration

Policy iteration repeats two steps until the policy stops changing.

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \cdots \xrightarrow{I} \pi_*$$

- **Evaluate (E)**: sweep the Bellman equation for v_π again and again until the values settle.
- **Improve (I)**: in every state, switch to the greedy action, i.e. the action with the largest $q_\pi(s, a)$.

On last week's slippery grid, the three evaluations took about 27, 12 and 10 sweeps.

Do We Have to Wait for the Values to Settle?

- The improvement step only asks **which action is best**, and rough values are often enough.
- So we do not wait. In every sweep, each state takes the value of its **best** action, not the value of the action its policy picks.
- This puts one evaluation sweep and one improvement step together. It is called **value iteration**.

Value Iteration

Policy evaluation from last week, for comparison:

$$v_{k+1}(s) \leftarrow \sum_{a} \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \, v_k(s') \right]$$

Value iteration: take the Bellman optimality equation and use it as an update rule.

$$v_{k+1}(s) \leftarrow \max_{a} \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \, v_k(s') \right]$$

- The only change is that $\sum_a \pi(a \mid s)$, which averages over the policy's actions, becomes $\max_a$, which takes the best action.
- No policy is kept during the sweeps.

> **Tip**
> Value iteration is policy iteration with every evaluation cut to a single sweep.

Value Iteration on the Grid With No Dice

We use last week's grid with no dice, $\gamma = 1$ and every value starting at 0. In each sweep, every square takes the best of its 4 moves, i.e. the points of the move + the value where it lands.

After sweep 1

−1	−1	−1	9
−1	−1	9	⭐
−1	−1	−1	9

After sweep 3

−3	7	8	9
2	8	9	⭐
−3	2	8	9

After sweep 5

6	7	8	9
5	8	9	⭐
1	2	8	9

After sweep 7 (final)

→ 6	→ 7	→ 8	↓ 9
↑ 5	→ 8	→ 9	⭐
↑ 4	← 3	→ 8	↑ 9

- The good news spreads out from the ⭐ by one square per sweep.
- In sweep 5, 🟢 drops the path through 💥 (worth 2) for the path over the top (worth 5). No improvement step was needed.
- Sweep 8 changes nothing, so we stop. These are the values and arrows that policy iteration gave last week.

The Value Iteration Algorithm

```
Input: small threshold θ > 0
Set V(s) = 0 for every state s, including the terminal states

repeat
    Δ = 0
    for each non-terminal state s:
        old = V(s)
        V(s) = max_a Σ_{s',r} p(s',r|s,a) [ r + γ V(s') ]
        Δ = max(Δ, |old − V(s)|)
until Δ < θ

π(s) = argmax_a Σ_{s',r} p(s',r|s,a) [ r + γ V(s') ]   for every state s
return π, V
```

- From any start, V gets closer to v_*. With $\gamma < 1$, each sweep shrinks the largest error by at least a factor γ, as in policy evaluation.
- The stop test is on the values. The greedy policy is often right much earlier.

Value Iteration on the Slippery Grid

Now the floor is slippery ($\frac{4}{6}$ the arrow and $\frac{1}{6}$ to each side), with $\gamma = 1$ and every value starting at 0.

After sweep 3

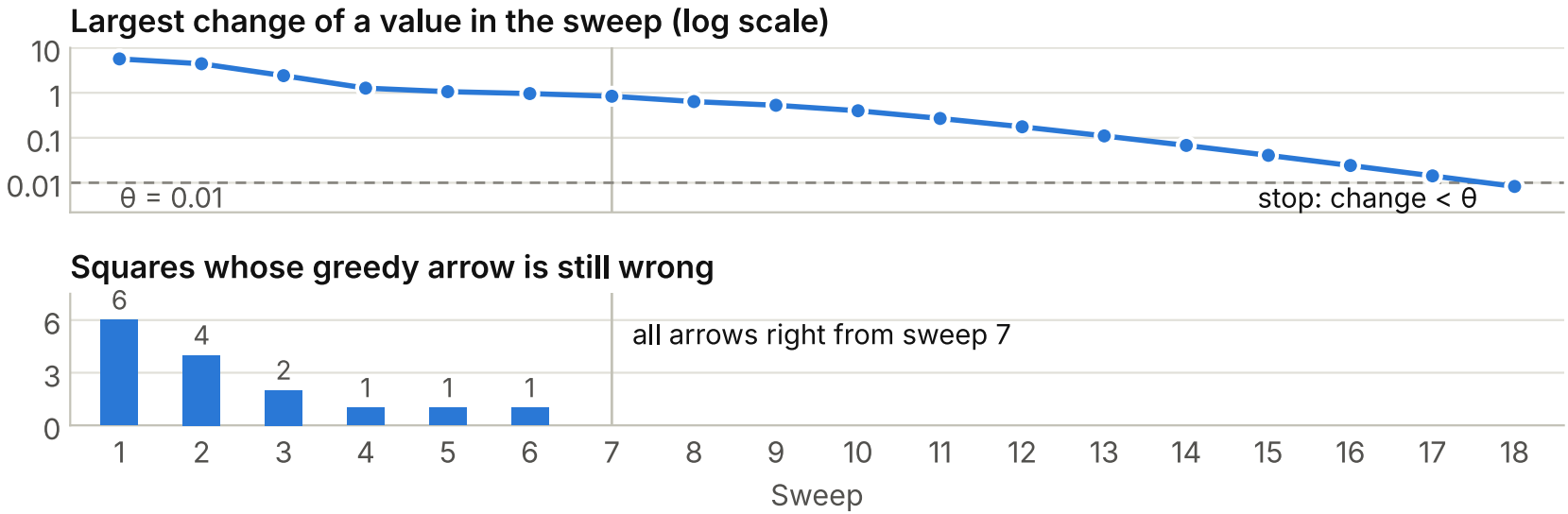
−3.0	0.4	4.6	7.3
−3.0	1.2	5.9	⭐
−3.0	−3.4	3.0	6.2

After sweep 18 (stop)

→ 1.9	→ 3.9	→ 6.6	↓ 8.1
↑ 0.1	→ 3.8	→ 6.7	⭐
↑ −1.5	→ −1.8	→ 4.5	↑ 6.7

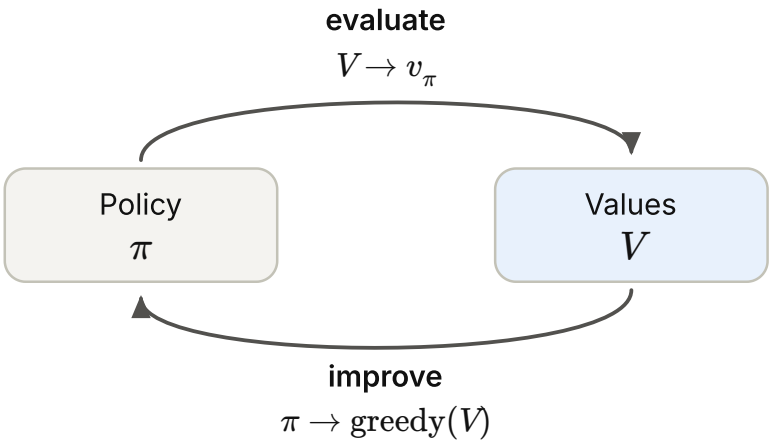
After sweep 18, no value changed by more than $\theta = 0.01$, so we stop. The values and arrows are the same as those of policy iteration last week.

When Should We Stop?



- All arrows are right from sweep 7, but the values keep changing until sweep 18.
- From sweep 4 to sweep 6 the arrows did not change, yet one of them was still wrong. So we cannot stop just because the arrows stopped changing. We stop on the values.

Generalized Policy Iteration (GPI)



When neither changes: $\pi = \pi_*$ and $V = v_*$

There is one policy π and one value table V. Evaluation updates V to match π and improvement makes π greedy with respect to V.

Method	Evaluation before each improvement
Policy iteration	Until the values settle
Value iteration	One sweep
MC control (this lecture)	One episode

Important
Most methods in this course follow GPI. They differ in how they evaluate.

What If Nobody Tells Us the Dice?

DP needs the model $p(s', r \mid s, a)$, e.g. the die of the slippery grid.

On a **new floor**, nobody knows the die.

What If Nobody Tells Us the Dice?

DP needs the model $p(s', r \mid s, a)$, e.g. the die of the slippery grid.

On a **new floor**, nobody knows the die.

We can still

1. let the robot follow its policy until it reaches the ⭐,
2. add up the rewards, which gives one return, and
3. repeat many times and take the average.

> **Important**
> Play and average. This is Monte Carlo.

Part 2: Monte Carlo Prediction

Learning From Episodes

The value is an expected return, so we can estimate it by averaging the returns we saw.

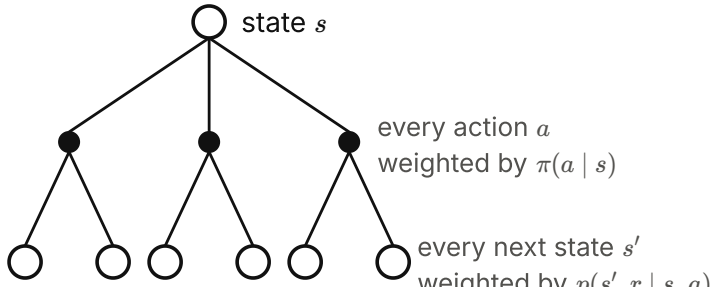
$$v_\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right] \approx \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_i(s)$$

- Play many episodes with π.
- Each time an episode visits s, write down the return that follows. $G_i(s)$ is the i-th such return and $N(s)$ counts them.
- The average gets closer to $v_\pi(s)$ as $N(s)$ grows.

MC needs **episodes that end**, because a return is known only at the end. It also needs a way to **play** π, in the real world or in a simulator.

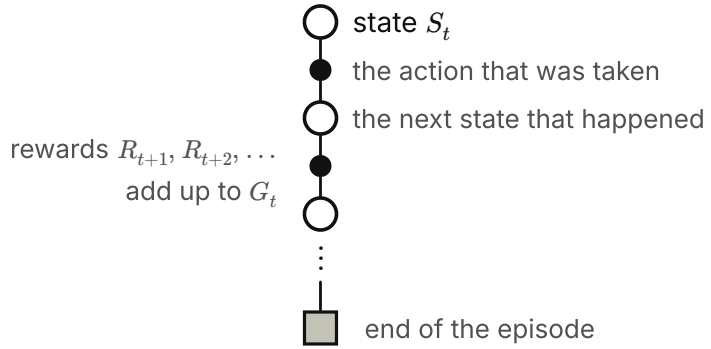
One Backup in DP and in MC

DP: one step, every branch



Needs the model p and the estimates $V(s')$

MC: one path, to the end of the episode



- DP moves $V(s)$ toward $\sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V(s')\right]$, which is an average over every branch.
- MC moves $V(s)$ toward the return G_t that happened. It uses only real rewards, so the estimate of one state does not depend on the estimates of other states.

Compute the Returns Backward

An episode gives
$S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T.$

Use $G_t = R_{t+1} + \gamma G_{t+1}$ and start from the end.

```
G = 0
for t = T-1, T-2, ..., 0:
    G = γ G + R_{t+1}
    # G is now the return from time t
```

One pass from the end gives every return in the episode.

Example with $\gamma = 1$:

$$A \xrightarrow{+2} B \xrightarrow{0} A \xrightarrow{+1} \text{end}$$

t	State	R_{t+1}	G_t
2	A	$+1$	1
1	B	0	$0 + 1 = 1$
0	A	$+2$	$2 + 1 = 3$

First-Visit and Every-Visit MC

A state can show up more than once in an episode. **First-visit MC** averages only the return after the first visit in each episode (in bold). **Every-visit MC** averages the returns after every visit.

	Path, with $\gamma = 1$	Returns of A	Returns of B
Episode 1	$A \xrightarrow{+2} B \xrightarrow{0} A \xrightarrow{+1} \text{end}$	**3**, 1	**1**
Episode 2	$B \xrightarrow{+3} A \xrightarrow{-1} B \xrightarrow{+2} \text{end}$	**1**	**4**, 2
First-visit V		$(3+1)/2 = 2$	$(1+4)/2 = 2.5$
Every-visit V		$(3+1+1)/3 \approx 1.67$	$(1+4+2)/3 \approx 2.33$

The Incremental Mean

We do not need to keep every return. The average of the first n returns comes from the average of the first $n-1$.

$$V_n = V_{n-1} + \frac{1}{n}\left(G_n - V_{n-1}\right)$$

Almost every update in RL has this shape.

$$\text{new} = \text{old} + \text{step size} \times (\text{target} - \text{old})$$

The every-visit returns of A were 3, 1 and 1.

n	G_n	Update	V_n
1	3	$0 + \frac{1}{1}(3 - 0)$	3
2	1	$3 + \frac{1}{2}(1 - 3)$	2
3	1	$2 + \frac{1}{3}(1 - 2)$	1.67

First-Visit MC Prediction

```
Input: policy π
Set V(s) = 0 and N(s) = 0 for every state s

repeat for many episodes:
    play one episode with π:  S_0, A_0, R_1, ..., S_{T-1}, A_{T-1}, R_T
    G = 0
    for t = T-1, T-2, ..., 0:
        G = γ G + R_{t+1}
        if S_t does not appear in S_0, ..., S_{t-1}:      # first visit
            N(S_t) = N(S_t) + 1
            V(S_t) = V(S_t) + (G - V(S_t)) / N(S_t)

return V   (close to v_π)
```

- For every-visit MC, drop the first-visit check and always update.
- No model anywhere. Only the states and rewards that we saw.

What MC Prediction Gives and Costs

Good	Bad
No model needed	Works only for episodes that end
Easy to code and easy to check	Learns only at the end of each episode
Can focus on the states we care about	Rare states get very few returns
No bias. Each first-visit return is an unbiased sample of $v_\pi(s)$	High variance. One return depends on every random step in the episode

You will see the variance with your own hands in the game at the end of the lecture.

Questions for the AlphaGo Paper

1. The value network is trained on z, which is the final result of a game. Which MC idea from today is this? Does it need $p(s', r \mid s, a)$?

2. Why take only one position from each self-play game? What would go wrong with all positions from the same game?

3. A rollout uses a weak fast policy. Whose value does it estimate, the strong player's or the fast policy's?

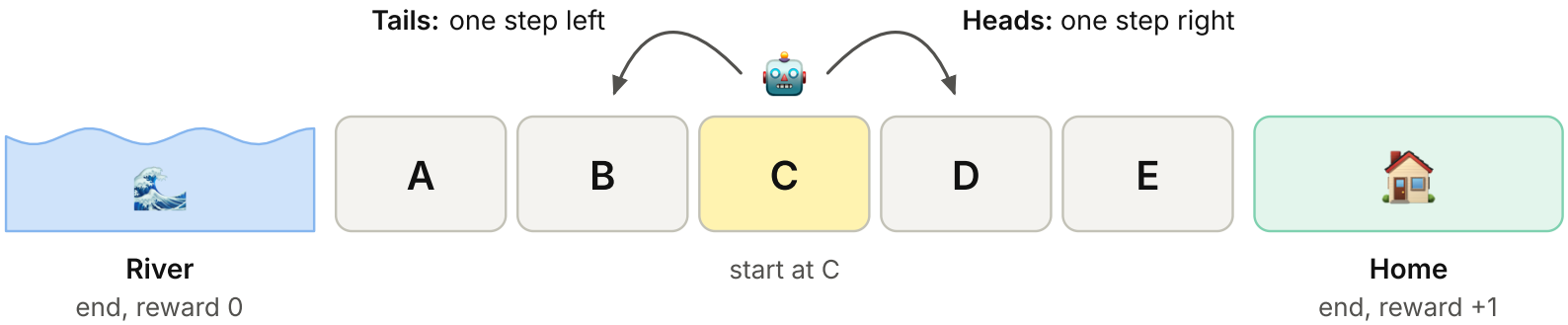
4. One rollout gives $+1$ or -1. About how many rollouts do you need to tell a position worth 0.1 from one worth 0.2?

5. Last week's sepsis paper also had a reward only at the end. Why could AlphaGo use MC targets while the sepsis paper could not play new episodes at all?

6. Which parts of AlphaGo would your project mentor need to trust before using a similar system in their field?

Practice: The Bridge Walk

The Bridge Walk



- Work in pairs. Each walk starts at C.
- Flip a coin and move the robot one square. Keep flipping until it reaches 🏠 or falls into 🌊.
- Every move gives 0 points, except the move into 🏠, which gives $+1$. Use $\gamma = 1$.
- Play **4 walks**.

Write Down Each Walk

For each walk, write the flips, the squares the robot stood on and how the walk ended.

Walk	Flips	Path	End	Reward at the end
1	T H H H H	C B C D E	🏠	$+1$
2	T T T	C B A	🌊	0
3				
4				

Run MC on Your Walks

1. For each walk, compute the return at every step by going backward. With $\gamma = 1$, what do you notice?
2. **First-visit**: for each square, count the walks that visited it and how many of those walks reached 🏠.
3. **Every-visit**: for each square, count all visits and how many of those visits were in walks that reached 🏠.
4. Divide to get V for each square, both ways.

With walks 1 and 2 only:

	A	B	C	D	E
First-visit: 🏠 / walks	0 / 1	1 / 2	1 / 2	1 / 1	1 / 1
Every-visit: 🏠 / visits	0 / 1	1 / 2	2 / 3	1 / 1	1 / 1

What Is Each Square Worth?

Each pair reads its counts and we add them up on the board.

	A	B	C	D	E
First-visit: 🏠 / walks					
Every-visit: 🏠 / visits					

- Which square has the most visits? Why?
- Which squares have the fewest first visits? Are their estimates more or less noisy?

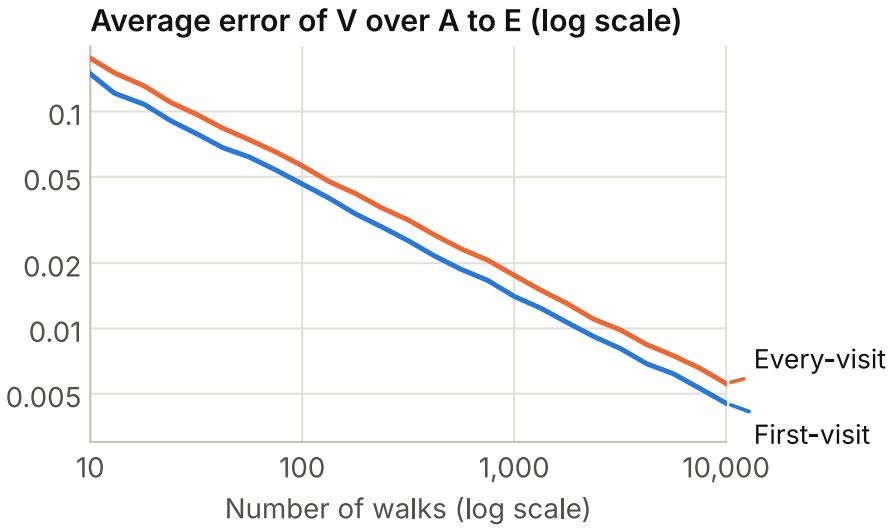
What the Computer Gets

The computer played two classes of 60 walks each and then 10,000 walks. First-visit estimates:

	A	B	C	D	E
Class 1 (60 walks)	0.23	0.37	0.55	0.69	0.89
Class 2 (60 walks)	0.12	0.36	0.53	0.70	0.89
10,000 walks	0.17	0.34	0.50	0.67	0.84
True value	**0.17**	**0.33**	**0.50**	**0.67**	**0.83**

- With 60 walks the classes disagree and miss the true values. With 10,000 walks they are close.
- The true values are $\frac{1}{6}, \frac{2}{6}, \ldots, \frac{5}{6}$, i.e. the chance that a walk from that square ends at 🏠.

How Fast Does the Error Shrink?



- The error shrinks like $1/\sqrt{N}$, i.e. 100 times more walks give about 10 times less error.
- Every-visit MC counts more returns, but the extra returns come from the same walks. Here its error is a bit higher.

Debrief

- Did your averages use the chance of heads anywhere?

- What was one walk?
- With $\gamma = 1$, what was the return at each step of a walk?
- Why did C get more every-visit counts than walks?

- Why did the class numbers differ from the true values?

- If the coin were not fair and nobody told you, what would change in your work?

Debrief

- Did your averages use the chance of heads anywhere? no, only the paths and how they ended. That is model-free
- What was one walk?
- With γ = 1, what was the return at each step of a walk?
- Why did C get more every-visit counts than walks?
- Why did the class numbers differ from the true values?
- If the coin were not fair and nobody told you, what would change in your work?

Debrief

- Did your averages use the chance of heads anywhere? no, only the paths and how they ended. That is model-free
- What was one walk? an episode, because it starts at C and always ends
- With γ = 1, what was the return at each step of a walk?
- Why did C get more every-visit counts than walks?
- Why did the class numbers differ from the true values?
- If the coin were not fair and nobody told you, what would change in your work?

Debrief

- Did your averages use the chance of heads anywhere? no, only the paths and how they ended. That is model-free
- What was one walk? an episode, because it starts at C and always ends
- With γ = 1, what was the return at each step of a walk? the final reward, i.e. 1 for 🏠 and 0 for 🌊
- Why did C get more every-visit counts than walks?
- Why did the class numbers differ from the true values?
- If the coin were not fair and nobody told you, what would change in your work?

Debrief

- Did your averages use the chance of heads anywhere? no, only the paths and how they ended. That is model-free
- What was one walk? an episode, because it starts at C and always ends
- With γ = 1, what was the return at each step of a walk? the final reward, i.e. 1 for 🏠 and 0 for 🌊
- Why did C get more every-visit counts than walks? the robot often comes back to C in the same walk
- Why did the class numbers differ from the true values?

- If the coin were not fair and nobody told you, what would change in your work?

Debrief

- Did your averages use the chance of heads anywhere? no, only the paths and how they ended. That is model-free
- What was one walk? an episode, because it starts at C and always ends
- With γ = 1, what was the return at each step of a walk? the final reward, i.e. 1 for 🏠 and 0 for 🌊
- Why did C get more every-visit counts than walks? the robot often comes back to C in the same walk
- Why did the class numbers differ from the true values? few walks. The error shrinks like 1 over the square root of the number of walks
- If the coin were not fair and nobody told you, what would change in your work?

Debrief

- Did your averages use the chance of heads anywhere? no, only the paths and how they ended. That is model-free
- What was one walk? an episode, because it starts at C and always ends
- With γ = 1, what was the return at each step of a walk? the final reward, i.e. 1 for 🏠 and 0 for 🌊
- Why did C get more every-visit counts than walks? the robot often comes back to C in the same walk
- Why did the class numbers differ from the true values? few walks. The error shrinks like 1 over the square root of the number of walks
- If the coin were not fair and nobody told you, what would change in your work? nothing. You would play and average in the same way

The Bridge Walk in Gymnasium

```python
import gymnasium as gym

class BridgeWalk(gym.Env):
    # 0 = river, 1 to 5 = A to E, 6 = home
    def __init__(self):
        self.observation_space = gym.spaces.Discrete(7)
        self.action_space = gym.spaces.Discrete(1)    # no choice, the coin decides

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        self.s = 3                                    # start at C
        return self.s, {}

    def step(self, action):
        self.s += 1 if self.np_random.random() < 0.5 else -1
        reward = 1.0 if self.s == 6 else 0.0
        terminated = self.s in (0, 6)
        return self.s, reward, terminated, False, {}
```

In the lab, you will run first-visit and every-visit MC on it and compare with the true values.

Common Mistakes

- Stopping value iteration as soon as the arrows stop changing. Stop on the values.
- Discounting the wrong term. The update is $G = \gamma G + R_{t+1}$, not $G = G + \gamma R_{t+1}$.
- Using R_t instead of R_{t+1}, i.e. the reward that led into the state.
- Updating V in the middle of an episode. MC knows G_t only at the end.
- Forgetting the first-visit check, which turns first-visit MC into every-visit MC.
- Trusting an estimate made from very few returns.
- Using MC on a task whose episodes may never end.

Practice Check

1. Write the value iteration update. What is the only change from policy evaluation?
2. On the slippery grid, the arrows did not change from sweep 4 to sweep 6, but one of them was still wrong. What does this tell you about when to stop?
3. For the episode $B \xrightarrow{+1} A \xrightarrow{+1} B \xrightarrow{-2} \text{end}$ with $\gamma = 0.5$, compute every return. Then give the first-visit and every-visit estimates of $V(B)$.
4. In the bridge walk, change the river reward to -1. What is the true value of C, and why?
5. Why does MC need episodes that end, while DP does not?
6. Your estimate of $V(A)$ comes from 4 returns. About how many returns do you need to cut its error in half?

Value iteration

Policy evaluation with a max, i.e. the Bellman optimality equation as an update

Stopping

Stop on the values, not when the arrows stop changing

GPI

Evaluation and improvement working on the same values and policy

Model-free

Learn values from played episodes, without $p(s', r \mid s, a)$

Monte Carlo

Estimate a value by averaging the returns of real episodes

Return, backward

$G = \gamma G + R_{t+1}$ from the last step to the first

First and every visit

Average the return after the first visit to a state or after every visit

Variance

One return is noisy. The error shrinks like $1/\sqrt{N}$

Before We Meet Again

1. **Read Sutton and Barto, Sections 4.4, 4.6 and 5.1**.

2. **Write a short response to the AlphaGo paper**, with a focus on the questions on the slide.

3. **Read the TD-Gammon paper for week 6**: Tesauro, Communications of the ACM 38(3), 1995.

Next lecture: Monte Carlo control, i.e. finding a good policy from played episodes, and then learning from each step without waiting for the end of the episode.

> **Tip**
> Ask yourself now if your project environment has episodes that end. If an episode can go on for a very long time, MC will learn slowly.

Learn More

Course Homepage